

Student 23 **PATAN MOHAMMED RIYAZ KHAN** 192572126RC

2 / 2 submitted

Q1. Student Grade Book Application

CODE

```

# Student Grade Book Application

def calculate_grade(average):
    if average >= 90:
        return 'O'
    elif average >= 80:
        return 'A+'
    elif average >= 70:
        return 'A'
    elif average >= 60:
        return 'B+'
    elif average >= 50:
        return 'B'
    else:
        return 'F'

# (a) Store student data - key = name, value = list of 5 subject marks
students = {
    "Aditi": [85, 90, 78, 92, 88],
    "Rahul": [65, 70, 55, 60, 72],
    "Sneha": [95, 92, 89, 96, 91],
    "Kiran": [45, 50, 40, 48, 52],
    "Meera": [78, 82, 75, 80, 79]
}

subjects = ["Maths", "Science", "English", "History", "Computer"]

# (b) Compute total, average, and grade using dictionary comprehension
results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": round(sum(marks) / len(marks), 2),
        "grade": calculate_grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}

# (c) Find the class topper (highest average)
topper = max(results, key=lambda name: results[name]["average"])

# Find the subject with the highest overall mean
subject_means = [
    sum(students[name][i] for name in students) / len(students)
    for i in range(len(subjects))
]
top_subject_index = subject_means.index(max(subject_means))
top_subject = subjects[top_subject_index]

# (d) Display formatted result table sorted by average in descending order
sorted_students = sorted(results.items(), key=lambda x: x[1]["average"], reverse=True)

print("=" * 70)
print("STUDENT GRADE BOOK REPORT")
print("=" * 70)
print(f"{'Name':<10}{'Marks':<30}{'Total':<8}{'Average':<10}{'Grade':<5}")
print("-" * 70)

for name, data in sorted_students:
    marks_str = str(data["marks"])
    print(f"{'Name':<10}{'marks_str':<30}{'data['total']':<8}{'data['average']':<10}{'data['grade']':<5}")

print("-" * 70)
print(f"\nClass Topper      : {topper} (Average: {results[topper]['average']})")
print(f"Best Performing Subject : {top_subject} (Mean: {round(max(subject_means), 2)})")
print("=" * 70)

```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

```
print("\nSubject-wise Mean Marks:")
for subj, mean in zip(subjects, subject_means):
    print(f" {subj:<10}: {round(mean, 2)}")
```

OUTPUT (no output)

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] >= amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')

def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)

create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

OUTPUT (no output)